Agentic AI: hello world to enterprise — Advanced

Portfolio projects · lessons 24–37

Each takes 1–3 days and ends with something to show an employer.

Project 1: Brightside support desk in production: three replicas, an eval gate and a load test

Goal: Ship the support desk as a three-replica service with a CI eval gate, per-ticket budgets and a measured load test that survives a rolling restart. A hiring manager can rerun the load log and check every number in the README.

Uses lessons: 28, 29, 32, 33, 34, 37

1. Start from app v3 (app/GUIDE.md, step 3), then run pytest on the 30 golden tasks and the 20 injection cases in data/support_eval.json under llm_replay; write the baseline task success and injection pass count into RESULTS.md.

2. Add the CI eval gate from lesson 28: the build fails if task success drops by more than 2 tasks below the baseline or if any injection case calls issue_refund; prove the gate works by committing one deliberately weakened prompt and recording the red build.

3. Wrap the agent in a FastAPI endpoint that streams the reply, add a /healthz route, write a Dockerfile on python:3.12-slim and a Compose file that runs three agent replicas behind the gateway from lesson 34.

4. Set the budgets in code: 6 steps per ticket, a $0.02 spend cap per ticket, a 20-second timeout per model call, tenacity retries with 3 attempts and exponential backoff with jitter capped at 8 seconds, and a token bucket of 5 requests per second with a burst of 10 on each replica.

5. Run the 300-ticket load script from lesson 34 at 10 concurrent requests against the three-replica fleet and save one log row per request: status, latency, steps, and cost from the OpenTelemetry JSON-lines traces.

6. Roll out a v2 prompt as a canary: send it to one replica for 10 percent of the golden set, compare success and cost with v1, and only then restart the other two replicas one at a time while the 300-ticket load keeps running.

7. Write RESULTS.md and README.md with p50 and p95 latency, error rate, cost per ticket, golden success, injection passes, and the number of requests that failed during the rolling restart (target: zero, with 429 and retry-after counted separately).

Deliverable: Git repo with app v3, the Dockerfile and Compose file for three replicas, the CI gate config, the load log as CSV, RESULTS.md and a README whose headline is p95 latency and cost per ticket against a stated target, with the table behind each number.

Stretch: Run the same load at 30 concurrent requests, find the replica count at which p95 first goes over 8 seconds, and report it as the capacity of one replica. · Give two tenants separate token buckets and show that a noisy tenant at 3 times its quota does not push the other tenant's p95 past the target.

Project 2: Brightside refund desk: a supervisor with two workers, a loop guard and an audit trail

Goal: Turn the single support agent into a supervisor with two workers, measure what the extra agents cost in tokens, reproduce a real loop between agents and stop it with guards, then prove every refund path leaves an audit row. The result is a decision backed by numbers, not a diagram.

Uses lessons: 23, 24, 25, 29, 30, 31

1. Measure the single agent on the 30 golden tasks in data/support_eval.json and record tokens in, tokens out, steps and wall-clock per task from the lesson 29 traces, along with task success.

2. Split the agent into a supervisor and two workers: an order worker with lookup_order and list_orders, and a policy worker with the returns policy and get_return_status; each worker keeps its own message history and sees only its own tools.

3. Limit what crosses a handoff: each worker returns a summary of at most 1,200 characters, and the supervisor never receives raw order rows from the CRM.

4. Compare the two versions on the same 30 tasks and write the tokens-per-task multiple and the success difference; keep the multi-agent version only if success rises by at least 2 tasks or wall-clock falls by at least 30 percent.

5. Reproduce a loop by making the policy worker ask the supervisor for more context on every turn; then add the lesson 25 guards: a handoff limit of 3, a shared budget of 8 steps for the whole run, and a state check that refuses a second refund for the same order_id.

6. Run the 20 injection cases through the multi-agent system and check that none of them reaches issue_refund; write one audit row per tool call (agent identity from lesson 31, tool, order_id, a hash of the arguments, timestamp) into the audit table that lesson 31 creates in data/31/audit.sqlite (copy the file first and leave the original untouched; columns ts, actor, on_behalf_of, tool, args, decision, reason, prev_hash, hash).

7. Write RESULTS.md with the single-versus-multi table, the loop before and after the guards, the injection count and the audit rows per ticket, and a README that ends with a one-paragraph decision on whether to ship the multi-agent version.

Deliverable: Git repo with the single agent, the supervisor and two workers, the guard code, the copy of the lesson 31 audit database, RESULTS.md and a README that shows the token multiple, the loop before and after, and the injection count.

Stretch: Route the policy worker to a smaller model with the routing rule from lesson 33 and report whether the success on the 30 tasks holds. · Replace the in-process call to the policy worker with an A2A call and measure the extra latency per task it adds.

Project 3: Brightside partner tracking over MCP and A2A, with OAuth and a failure drill

Goal: Expose the desk's order tools as a scoped MCP server, let the desk discover and call a carrier's agent through its agent card, and prove that a failed partner call never leaves a ticket stuck. Partner integrations are where enterprise agents fail, so this one is measured on failure as well as success.

Uses lessons: 21, 26, 27, 27a, 30

1. Expose lookup_order and get_return_status as an MCP server over Streamable HTTP on localhost, connect an MCP client, list the tools and record each description length; every description must stay under 200 characters.

2. Put OAuth in front of the server with the local token issuer built with Authlib in lesson 26: tokens carry orders:read or orders:write, and six tests show that a read-only token can call lookup_order and is refused by issue_refund.

3. Add a tool-poisoning check that rejects any tool description over 200 characters or containing the words ignore, send or email; test it on 3 descriptions you write to poison the tool and on the 2 real ones, where all 3 poisoned descriptions must be rejected and both real ones accepted.

4. Publish an agent card for a fictional carrier, Northline Parcels, at the well-known URL with a name, a version, one skill called track_parcel and bearer-token auth required.

5. From the desk agent, fetch the card, check the skill and the auth requirement before sending anything, then send a tracking task and follow its states (submitted, working, input-required, completed) while streaming its updates.

6. Stop the carrier server in the middle of 5 of 20 tracking tasks and check that the desk marks each of those tickets escalated with the reason carrier_unavailable; no ticket may stay in working after the run.

7. Write README.md with the call sequence, the scope-by-tool auth matrix, and the results: MCP calls and A2A round trips per question, median and p95 latency, and the count of tasks in each terminal state.

Deliverable: Git repo with the MCP server, the OAuth test file, the poisoning check and its test cases, the carrier agent and its card, the desk client, and a README with the auth matrix and the terminal-state count from the failure drill.

Stretch: Make the carrier ask for a postcode mid-task (input-required), have the desk answer, and show the task resumes and completes with the same task id. · Revoke a token during a session and show that the next call is refused within one request, with the refusal written to the audit log.

Project 4: Brightside supplier invoices: a durable payment run that survives a worker kill

Goal: Apply the same production controls to a different job, accounts payable at Brightside, where paying an invoice twice costs real money. The payment run is killed five times on purpose, and the ledger must show each invoice paid exactly once to the cent.

Uses lessons: 14, 19, 30, 32, 35, 36

1. Generate 200 supplier invoices with a seeded script (seed 11) from 25 fictional suppliers with name@example.com contacts; each has an invoice number, a purchase order id, a goods-receipt id, an amount and a currency, and 15 of them are mismatches (price off by more than 2 percent, missing receipt, duplicate invoice number).

2. Model each invoice as a Temporal workflow with four steps: three-way match, a human approval wait for amounts of $500 or more (the approval gate from lesson 14), pay, and record; run the Temporal dev server and the worker with Docker Compose.

3. Give the pay step an idempotency key made from invoice_id and amount, and write each payment to a SQLite ledger table with a unique constraint on that key; the payment call is a stub that writes to the ledger and moves no money.

4. Run the payment run for all 200 invoices with 4 workers and record the total time and the final state of every invoice: paid, rejected, or waiting for approval.

5. Run the kill drill: five times during the run, kill the worker process at a seeded random moment, restart it and let Temporal resume; after each kill, check that every approved invoice has exactly one ledger row and that the run finishes within 10 minutes of the restart.

6. Repeat the same five kills on a naive version with no idempotency key and no durable run (a plain loop with retries), and count the duplicate payments it makes.

7. Write RESULTS.md with the drill table (kill point, invoices resumed, duplicate payments in each version) and README.md with the match rule, the approval rule and the ledger reconciliation: the sum of paid equals the sum of approved amounts to the cent, computed with Decimal.

Deliverable: Git repo with the invoice generator and seed, the Temporal workflow and worker, the Compose file, the ledger schema, the naive comparison, RESULTS.md and a README that shows the drill table and the reconciliation result.

Stretch: Hold a payment when a supplier's bank details change during a run, and run 5 injection cases where invoice text tries to change the bank account; show that none of them changes it. · Raise the worker count from 4 to 16 and report at which count the ledger write becomes the bottleneck.